Bank Agent Demo Reset Runbook

Repeatable reset, execution, human approval, and verification procedure

Oracle Cloud Infrastructure (OCI) | Autonomous Database | Oracle VECTOR | Ollama

Host	bank-agent-poc
Linux user	opc
Application directory	/home/opc/bank-agent
Database application user	BANKAPP
Demo employee	CSR001 / CUSTOMER_SERVICE
Demo transaction	847291
Generation model	qwen2.5:3b
Embedding model	nomic-embed-text

Purpose: Reset only demo-generated action and audit records so the banking agent can be demonstrated repeatedly from a clean state. Source banking records, fraud data, policies, and vector embeddings are preserved.

Security boundary: The reset runs as Linux user opc and connects to Autonomous Database as BANKAPP using values loaded from .env. Passwords are never displayed by the script.

Runbook version 1.0 | Live banking agent demonstration

1. Scope and Safety

This runbook resets the transient state created by a completed Bank Agent demonstration. The reset is intentionally narrow and transaction-scoped.

What the reset deletes

- ACTION_REQUESTS rows for transaction 847291, or the transaction ID explicitly supplied to the script.
- AI_AUDIT_LOG rows for the same transaction.

What the reset preserves

- CUSTOMERS
- ACCOUNTS
- TRANSACTIONS
- FRAUD_EVENTS
- POLICY_CHUNKS
- Oracle VECTOR embeddings

Do not run as root: The supported operator is the opc Linux user. The script verifies the current OS user and exits if it is not opc.

Do not expose credentials: The .env file contains database and wallet secrets. Never print the database password or wallet password during the demo. Keep .env mode 600.

2. Preconditions

1. SSH to the OCI compute instance bank-agent-poc as opc.
2. Change to /home/opc/bank-agent.
3. Confirm .venv, .env, wallet, bank_agent.py, review_action.py, and search_policy.py exist.
4. Confirm the database connection and Ollama models were previously validated.

```
cd /home/opc/bank-agent
pwd
whoami
ls -al

# Expected:
# /home/opc/bank-agent
# opc
```

3. Standard Session Startup

```
cd /home/opc/bank-agent
source .venv/bin/activate
set -a
source .env
set +a
```

Verify non-secret configuration values:

```
echo "$DB_USER"
echo "$DB_DSN"
echo "$EMPLOYEE_ID"
echo "$EMPLOYEE_ROLE"
echo "$GEN_MODEL"
echo "$EMBED_MODEL"
```

4. One-Time Installation of the Reset Script

Perform this section only once unless reset_demo.sh is intentionally replaced. Run every command as Linux user opc from /home/opc/bank-agent.

4.1 Create reset_demo.sh

```
cat > reset_demo.sh <<'SH'
# Paste the complete script from Appendix A here.
SH
```

4.2 Make it executable

```
chmod 700 reset_demo.sh
ls -l reset_demo.sh
```

Expected permissions: -rwx------ with opc ownership.

5. Reset Procedure Before Every Live Demo

1. Confirm you are on bank-agent-poc as opc and in /home/opc/bank-agent.

2. Activate .venv and load .env if this is a new SSH session.

3. Run the reset script: ./reset_demo.sh

4. When prompted, type RESET exactly.

5. Confirm Action requests remaining = 0 and Audit records remaining = 0, while source data remains present.

```
./reset_demo.sh

# Prompt:
Type RESET to continue: RESET
```

Expected successful reset

```
[3] Current generated demo state:
    Action requests : 1
    Audit records   : 8

[4] Generated demo state removed.
    Action requests deleted : 1
    Audit records deleted   : 8

[5] Reset verification:
    Action requests remaining : 0
    Audit records remaining   : 0

[6] Source-data verification:
    Transaction        : PRESERVED
    Related account    : 1
    Fraud events       : 1
    Embedded policies  : 3

RESET COMPLETE
```

Identity values: Oracle identity columns are not reset. A later Action ID may be 2, 3, 4, or higher. That is expected and preserves database identity semantics.

6. Live Demo Execution Sequence

6.1 Start the banking agent

```
python bank_agent.py
```

```
[1] Transaction evidence retrieved
[2] Account evidence retrieved
[3] Authorized fraud evidence retrieved (1 event(s))
[4] Authorized policy retrieval completed (2 chunk(s))
    - CV-04 Card Verification Procedure
    - CA-12 Card Account Status Procedure
[5] Grounded prompt assembled
[6] AI response generated
[7] Action request <ID> created with status PENDING
```

6.2 Presentation pause - demonstrate the control boundary

Suggested narration: The agent has completed investigation, retrieved only authorized evidence, interpreted policy, and proposed an action. It has not executed or approved the action. The action remains PENDING until a human reviews it.

6.3 Perform human review

```
python review_action.py
```

At the prompt, enter APPROVE, REJECT, or SKIP.

```
Type APPROVE, REJECT, or SKIP: APPROVE
```

6.4 Expected approved state

```
Action <ID> is now APPROVED.
Approval status : APPROVED
Approved by     : CSR001
Approved at     : <timestamp>
```

7. Verification After the Demo

Use the following read-only verification as opc. Python connects as BANKAPP through .env and the mTLS wallet.

```
python - <<'PY'
import os
import oracledb

conn = oracledb.connect(
    user=os.environ['DB_USER'],
    password=os.environ['DB_PASSWORD'],
    dsn=os.environ['DB_DSN'],
    config_dir=os.environ['WALLET_DIR'],
    wallet_location=os.environ['WALLET_DIR'],
    wallet_password=os.environ['WALLET_PASSWORD']
)
cur = conn.cursor()

print('\n=== ACTION REQUESTS ===\n')
cur.execute('''
    SELECT action_id, transaction_id, employee_id,
           proposed_action, approval_status,
           approved_by, created_at, approved_at
    FROM action_requests
    WHERE transaction_id = 847291
    ORDER BY action_id
''')
for row in cur:
    print(row)

print('\n=== AI AUDIT TRAIL ===\n')
cur.execute('''
    SELECT audit_id, event_time, employee_id,
           employee_role, stage, outcome
    FROM ai_audit_log
    WHERE transaction_id = 847291
    ORDER BY audit_id
''')
for row in cur:
    print(row)
```

```
cur.close()
conn.close()
PY
```

Expected audit sequence

```
TRANSACTION_LOOKUP  -> SUCCESS
ACCOUNT_LOOKUP      -> SUCCESS
FRAUD_LOOKUP        -> SAFE_DATA_ONLY
POLICY_RETRIEVAL    -> AUTHORIZED_ONLY
PROMPT_ASSEMBLY     -> GROUNDED
LLM_RESPONSE        -> GENERATED
ACTION_REQUEST      -> PENDING_HUMAN_APPROVAL
HUMAN_REVIEW        -> APPROVED or REJECTED
```

8. Troubleshooting

Wrong Linux user

```
whoami
# Expected: opc
```

Reconnect as opc. Do not bypass the user check and do not run the application as root.

python-oracledb import failure

```
source /home/opc/bank-agent/.venv/bin/activate
python -c "import oracledb; print(oracledb.__version__)"
```

Database connection failure

```
python test_db.py
```

Confirm .env is loaded, WALLET_DIR is /home/opc/bank-agent/wallet, and the wallet password is correct. Do not display the password.

Ollama model mismatch

```
ollama list
echo "$GEN_MODEL"
echo "$EMBED_MODEL"
```

Expected: qwen2.5:3b for generation and nomic-embed-text for embeddings.

No PENDING action exists

Run bank_agent.py first. review_action.py is intentionally unable to approve an action the agent has not created.

Action ID does not return to 1

Expected. The reset deletes rows but does not reset Oracle identity sequences.

9. Operator Quick Reference

```
# NEW SSH SESSION
cd /home/opc/bank-agent
source .venv/bin/activate
set -a
source .env
set +a

# RESET PRIOR DEMO STATE
./reset_demo.sh
# Type: RESET

# RUN THE AGENT
python bank_agent.py
# Confirm proposed action is PENDING

# HUMAN DECISION
```

```
python review_action.py
# Type: APPROVE, REJECT, or SKIP

# RESET AGAIN BEFORE NEXT DEMO
./reset_demo.sh
```

Presentation objective: Demonstrate that agentic AI can increase employee capacity without removing human accountability: investigate automatically, retrieve authorized context, generate grounded decision support, create a pending action, and require an explicit human decision.

Appendix A - Complete reset_demo.sh

Create this file as Linux user opc in /home/opc/bank-agent. Comments are intentionally verbose so the operator can explain each control during a live walkthrough.

```
#!/usr/bin/env bash

# ======================================================================
# BANK AGENT DEMO RESET SCRIPT
# ======================================================================
# PURPOSE:
# Reset generated demo state for one transaction without changing source
# banking data, fraud events, policies, or vector embeddings.
#
# RUN ON:
# Host: bank-agent-poc | Linux user: opc | /home/opc/bank-agent
# Database user: BANKAPP, loaded from .env
#
# DELETES ONLY:
#   - ACTION_REQUESTS for the selected transaction
#   - AI_AUDIT_LOG for the selected transaction
#
# PRESERVES:
#   - CUSTOMERS, ACCOUNTS, TRANSACTIONS, FRAUD_EVENTS
#   - POLICY_CHUNKS and vector embeddings
#
# SECURITY:
# Passwords are loaded from .env and never printed.
# ======================================================================

# Fail on command errors, undefined variables, and pipeline failures.
set -euo pipefail

# Use 847291 unless a different transaction ID is supplied.
TRANSACTION_ID="${1:-847291}"
APP_DIR="/home/opc/bank-agent"
VENV_DIR="${APP_DIR}/.venv"
ENV_FILE="${APP_DIR}/.env"

echo
echo "============================================================"
echo " BANK AGENT DEMO RESET"
echo "============================================================"
echo

# STEP 1 - Verify the non-root operating-system user.
CURRENT_USER="$(whoami)"
echo "[CHECK] Linux user       : ${CURRENT_USER}"
if [[ "${CURRENT_USER}" != "opc" ]]; then
    echo "ERROR: Run this script as Linux user 'opc'."
    exit 1
fi

# STEP 2 - Verify and enter the application directory.
echo "[CHECK] Application path : ${APP_DIR}"
if [[ ! -d "${APP_DIR}" ]]; then
    echo "ERROR: Application directory does not exist: ${APP_DIR}"
    exit 1
fi
cd "${APP_DIR}"

# STEP 3 - Verify and activate the Python virtual environment.
if [[ ! -f "${VENV_DIR}/bin/activate" ]]; then
    echo "ERROR: Python virtual environment not found: ${VENV_DIR}"
    exit 1
fi
echo "[CHECK] Python venv      : FOUND"
source "${VENV_DIR}/bin/activate"
echo "[CHECK] Python           : $(which python)"

# STEP 4 - Verify and load environment variables from .env.
if [[ ! -f "${ENV_FILE}" ]]; then
    echo "ERROR: Environment file not found: ${ENV_FILE}"
    exit 1
fi
echo "[CHECK] Environment file : FOUND"
set -a
source "${ENV_FILE}"
set +a
echo "[CHECK] Environment      : LOADED"

# STEP 5 - Check required variables without printing secrets.
REQUIRED_VARIABLES=(
  DB_USER DB_PASSWORD DB_DSN WALLET_DIR WALLET_PASSWORD
  EMPLOYEE_ID EMPLOYEE_ROLE GEN_MODEL EMBED_MODEL
)
for VARIABLE in "${REQUIRED_VARIABLES[@]}"; do
    if [[ -z "${!VARIABLE:-}" ]]; then
        echo "ERROR: Required variable ${VARIABLE} is not set."
```

```
        exit 1
    fi
done

echo "[CHECK] Required vars    : PRESENT"
echo "Database user            : ${DB_USER}"
echo "Database service         : ${DB_DSN}"
echo "Employee                 : ${EMPLOYEE_ID}"
echo "Employee role            : ${EMPLOYEE_ROLE}"
echo "Transaction to reset     : ${TRANSACTION_ID}"
echo "Passwords                 : [NOT DISPLAYED]"

# STEP 6 - Require an explicit operator confirmation before deletes.
echo
echo "This will DELETE previous ACTION_REQUESTS and AI_AUDIT_LOG"
echo "records for transaction ${TRANSACTION_ID}."
echo "Source banking data, policies, and vectors will be preserved."
echo
read -r -p "Type RESET to continue: " CONFIRMATION
if [[ "${CONFIRMATION}" != "RESET" ]]; then
    echo "Reset cancelled."
    exit 0
fi

# Export the selected transaction ID to the embedded Python program.
export DEMO_TRANSACTION_ID="${TRANSACTION_ID}"

# STEP 7 - Connect to Oracle and reset the generated demo state.
python - <<'PYCODE'
import os
import sys
import oracledb

transaction_id = int(os.environ["DEMO_TRANSACTION_ID"])

print("\n================================================================")
print(" DATABASE RESET")
print("================================================================\n")

try:
    conn = oracledb.connect(
        user=os.environ["DB_USER"],
        password=os.environ["DB_PASSWORD"],
        dsn=os.environ["DB_DSN"],
        config_dir=os.environ["WALLET_DIR"],
        wallet_location=os.environ["WALLET_DIR"],
        wallet_password=os.environ["WALLET_PASSWORD"]
    )
except Exception as exc:
    print("ERROR: Could not connect to Oracle Database.")
    print(exc)
    sys.exit(1)

cur = conn.cursor()
print("[1] Oracle connection established.")
print("    User        :", os.environ["DB_USER"])
print("    Transaction :", transaction_id)

# Verify the source transaction exists before deleting anything.
cur.execute("""
    SELECT transaction_id, account_id, merchant, amount, status, decline_code
    FROM transactions
    WHERE transaction_id = :transaction_id
""", transaction_id=transaction_id)
transaction = cur.fetchone()
if transaction is None:
    print(f"ERROR: Transaction {transaction_id} does not exist.")
    cur.close(); conn.close(); sys.exit(1)

print("\n[2] Demo transaction verified.")
print("    Transaction ID :", transaction[0])
print("    Account ID     :", transaction[1])
print("    Merchant       :", transaction[2])
print("    Amount         :", transaction[3])
print("    Status         :", transaction[4])
print("    Decline code   :", transaction[5])

# Count generated records before reset.
cur.execute("SELECT COUNT(*) FROM action_requests WHERE transaction_id=:id", id=transaction_id)
actions_before = cur.fetchone()[0]
cur.execute("SELECT COUNT(*) FROM ai_audit_log WHERE transaction_id=:id", id=transaction_id)
audit_before = cur.fetchone()[0]
print("\n[3] Current generated demo state:")
print("    Action requests :", actions_before)
print("    Audit records   :", audit_before)

# Delete only generated state for this transaction.
cur.execute("DELETE FROM action_requests WHERE transaction_id=:id", id=transaction_id)
actions_deleted = cur.rowcount
cur.execute("DELETE FROM ai_audit_log WHERE transaction_id=:id", id=transaction_id)
audit_deleted = cur.rowcount
conn.commit()
print("\n[4] Generated demo state removed.")
print("    Action requests deleted :", actions_deleted)
print("    Audit records deleted   :", audit_deleted)

# Verify generated records are now empty.
cur.execute("SELECT COUNT(*) FROM action_requests WHERE transaction_id=:id", id=transaction_id)
actions_after = cur.fetchone()[0]
cur.execute("SELECT COUNT(*) FROM ai_audit_log WHERE transaction_id=:id", id=transaction_id)
audit_after = cur.fetchone()[0]
print("\n[5] Reset verification:")
print("    Action requests remaining :", actions_after)
print("    Audit records remaining   :", audit_after)

# Verify required source data remains intact.
cur.execute("""
    SELECT COUNT(*) FROM accounts a
    JOIN transactions t ON t.account_id=a.account_id
    WHERE t.transaction_id=:id
```

```
""", id=transaction_id)
account_count = cur.fetchone()[0]
cur.execute("SELECT COUNT(*) FROM fraud_events WHERE transaction_id=:id", id=transaction_id)
fraud_count = cur.fetchone()[0]
cur.execute("SELECT COUNT(*) FROM policy_chunks WHERE embedding IS NOT NULL")
policy_vector_count = cur.fetchone()[0]
print("\n[6] Source-data verification:")
print("    Transaction        : PRESERVED")
print("    Related account    :", account_count)
print("    Fraud events       :", fraud_count)
print("    Embedded policies  :", policy_vector_count)


if actions_after != 0 or audit_after != 0:
    print("WARNING: Reset did not produce an empty generated state.")
    cur.close(); conn.close(); sys.exit(1)


cur.close(); conn.close()
print("\n============================================================")
print(" RESET COMPLETE")
print("============================================================")
print(f"Transaction {transaction_id} is ready for another demo.\n")
PYCODE


# STEP 8 - Remind the operator what to run next.
echo "NEXT DEMO STEPS"
echo "1. python bank_agent.py"
echo "2. Confirm the new action is PENDING."
echo "3. python review_action.py"
echo "4. Type APPROVE or REJECT when prompted."
```

Appendix B - Demo Architecture and Control Narrative

```
Employee question
      |
      v
Bank AI Agent
      |
      +--> Transaction evidence
      +--> Account evidence
      +--> Safe fraud evidence only
      +--> Oracle VECTOR semantic retrieval
                |
                +--> CUSTOMER_SERVICE role filter
                +--> CV-04 / CA-12 allowed
                +--> FR-09 restricted
      |
      v
Grounded qwen2.5:3b response
      |
      v
ACTION_REQUEST = PENDING
      |
      X  AI cannot approve
      |
      v
Human employee
      |
      +--> APPROVE / REJECT / SKIP
      |
      v
AI_AUDIT_LOG records chronology
```

Control points demonstrated

- Least-privilege retrieval: restricted fraud fields are not selected for CUSTOMER_SERVICE.
- Role-filtered RAG: Oracle filters authorized policy chunks before they reach the LLM.
- Grounding: the model receives transaction, account, safe fraud, and authorized policy evidence.
- Deterministic action creation: the LLM does not directly execute database state changes.
- Human-in-the-loop: the proposed action remains PENDING until an explicit human decision.
- Auditability: AI stages and the later human review are recorded as distinct events.

Recommended demo close

Key message: Agentic AI can increase employee capacity without removing human accountability. The system automates investigation and evidence assembly, but authorization is enforced outside the model and the final controlled action remains a human decision.